11. Generics and Error Handling

Task 1:

Code:

```
import Foundation
struct Stack<Element> {
    private var items: [Element] = []
    mutating func push(_ item: Element) {
        items.append(item)
    }
    mutating func pop() -> Element? {
        return items.popLast()
    }
    var top: Element? {
        return items.last
    }
    var isEmpty: Bool {
        return items.isEmpty
    }
    var count: Int {
        return items.count
    }
}
var intStack = Stack<Int>()
intStack.push(10)
intStack.push(20)
print("Integer top:", intStack.top ?? 0)
print("Integer pop:", intStack.pop() ?? 0)
```

```
print("Integer count:", intStack.count)
var stringStack = Stack<String>()
stringStack.push("Swift")
stringStack.push("Java")
print("String top:", stringStack.top ?? "Empty")
print("String pop:", stringStack.pop() ?? "Empty")
print("String count:", stringStack.count)
func largest<T: Comparable>(in items: [T]) -> T? {
   return items.max()
}
print("Largest integer:", largest(in: [10, 40, 20]) ?? 0)
print("Largest string:", largest(in: ["Apple", "Orange", "Banana"]) ?? "None")
func swapValues<T>(_ a: inout T, _ b: inout T) {
   let temp = a
   a = b
   b = temp
}
var x = 10
var y = 20
swapValues(&x, &y)
print("After swap:", x, y)
```

Output:

```
Output

Integer top: 20
Integer pop: 20
Integer count: 1
String top: Java
String pop: Java
String count: 1
Largest integer: 40
Largest string: Orange
After swap: 20 10
```

Task 2:

Code:

```
import Foundation
enum ATMError: Error {
    case insufficientFunds(shortBy: Double)
    case invalidAmount
    case dailyLimitExceeded
}
struct Account {
    var balance: Double
    let dailyLimit: Double = 20000
    mutating func withdraw(_ amount: Double) throws {
        if amount <= 0 {
            throw ATMError.invalidAmount
        }
        if amount > dailyLimit {
            throw ATMError.dailyLimitExceeded
```

```
        }
        if amount > balance {
            throw ATMError.insufficientFunds(
                shortBy: amount - balance
            )
        }
        balance -= amount
        print("Withdrawal successful: \(amount)")
        print("Remaining balance: \(balance)")
    }
}
var account = Account(balance: 10000)
let amounts: [Double] = [500, -10, 25000, 12000]
for amount in amounts {
    do {
        try account.withdraw(amount)
    } catch ATMError.invalidAmount {
        print("Error: Invalid withdrawal amount")
    } catch ATMError.dailyLimitExceeded {
        print("Error: Daily limit exceeded")
    } catch ATMError.insufficientFunds(let shortBy) {
        print("Error: Insufficient funds. Short by \(shortBy)")
    } catch {
        print("Unknown error")
    }
}
let result: Void? = try? account.withdraw(100)
```

print("Withdrawal result:", result as Any)

Output:

```
Output

ERROR!
Withdrawal successful: 500.0
Remaining balance: 9500.0
Error: Invalid withdrawal amount
Error: Daily limit exceeded
Error: Insufficient funds. Short by 2500.0
Withdrawal successful: 100.0
Remaining balance: 9400.0
Withdrawal result: Optional(())
```

Task 3:

Code:

```
import Foundation

enum AgeError: Error {
    case empty
    case notANumber
    case outOfRange
}

func parseAge(_ text: String) throws -> Int {
    let trimmed = text.trimmingCharacters(in: .whitespaces)
    if trimmed.isEmpty {
        throw AgeError.empty
    }
    guard let age = Int(trimmed) else {
        throw AgeError.notANumber
    }
```

```
    guard (1...120).contains(age) else {
        throw AgeError.outOfRange
    }
    return age
}
let result = Result {
    try parseAge("25")
}
switch result {
case .success(let age):
    print("Valid age:", age)
case .failure(let error):
    switch error {
    case AgeError.empty:
        print("Error: Age cannot be empty")
    case AgeError.notANumber:
        print("Error: Enter a valid number")
    case AgeError.outOfRange:
        print("Error: Age must be between 1 and 120")
    default:
        print("Unknown error")
    }
}
func saveReport() {
    print("Opening file")
    defer {
        print("Closing file")
```

```
    }
    print("Saving report")
}
saveReport()
```

Output:

```
Output

Valid age: 25
Opening file
Saving report
Closing file
```